

100 Easy to Medium SQL Problems on EACH SQL Concepts

Query Optimization & Performance Improvement Plan

- Use the correct **JOIN** for query optimization.
 - Select only the necessary columns.
 - Analyze query execution using the **EXPLAIN** plan.
 - Index the most frequently used columns.
 - Implement composite indexing for columns commonly used together.
 - Continuously monitor performance for improvements.
-

SQL Concepts and Practice Problems

Here's a comprehensive list of essential SQL concepts, each accompanied by five practice questions to help students build their SQL skills.

1. Basic SQL Queries

- **Concept:** Writing simple queries to retrieve data from tables.
 - **Questions:**
 1. Select all columns from the **sales** table.
 2. Retrieve only product names from the **product** table.
 3. Select the first 10 records from the **sales** table.
 4. Find distinct categories in the **category** table.
 5. Display all stores located in Canada.
-

2. Filtering Data

- **Concept:** Using the **WHERE** clause to filter records.
- **Questions:**
 1. Retrieve sales that occurred after January 1, 2023.
 2. Filter products with a price greater than \$500.
 3. Find warranty claims with the status "Warranty Void."

4. Retrieve products launched in the past year.
 5. Select sales from store ID "ST-70."
-

3. Aggregate Functions

- **Concept:** Using **SUM**, **COUNT**, **AVG**, etc., to aggregate data.
 - **Questions:**
 1. Count the total number of units sold in the **sales** table.
 2. Calculate the average product price in the **product** table.
 3. Find the total revenue generated from all sales.
 4. Determine the highest-priced product.
 5. Count the number of warranty claims in the **warranty** table.
-

4. Grouping Data

- **Concept:** Using the **GROUP BY** clause to group results.
 - **Questions:**
 1. Find total sales by each store.
 2. Group warranty claims by their repair status.
 3. Find the total units sold for each product category.
 4. Count the number of products in each category.
 5. Calculate total revenue generated by each product in the last year.
-

5. Joining Tables

- **Concept:** Using **JOIN** to combine data from multiple tables.
 - **Questions:**
 1. Join **sales** and **product** tables to display sales with product names.
 2. Join **warranty** and **sales** tables to find claims related to specific sales.
 3. Display products along with their categories.
 4. Retrieve sales information along with store details.
 5. Show warranty claims with corresponding product names.
-

6. Subqueries

- **Concept:** Writing queries inside other queries for complex operations.

- **Questions:**
 1. Find products that have never been sold using a subquery.
 2. Use a subquery to find the average price of products sold.
 3. Retrieve stores with sales greater than the average across all stores.
 4. Find warranty claims for products sold more than once.
 5. Use a subquery to find the total units sold for a specific product.
-

7. Date Functions

- **Concept:** Manipulating and filtering date values in queries.
 - **Questions:**
 1. Retrieve sales from the last 30 days.
 2. Extract the year from the `sales_date`.
 3. Find warranty claims filed in 2020.
 4. Calculate the number of days between `sales_date` and today.
 5. Group sales by month and year.
-

8. String Functions

- **Concept:** Manipulating strings within SQL queries.
 - **Questions:**
 1. Concatenate customer first and last names.
 2. Find product names that contain "iPhone."
 3. Calculate the length of each product name.
 4. Convert product names to uppercase.
 5. Extract a substring from the product name.
-

9. Case Statements

- **Concept:** Using `CASE` to add conditional logic to queries.
- **Questions:**
 1. Classify products based on price ranges (e.g., "Budget," "Mid-range," "Premium").
 2. Create a status column for warranty claims based on repair status.
 3. Assign a sales category based on units sold.
 4. Use `CASE` to display different messages based on `sales_date`.
 5. Categorize stores based on total units sold.

10. Window Functions

- **Concept:** Performing calculations across a set of rows related to the current row.
- **Questions:**
 1. Calculate the running total of units sold per store.
 2. Rank products by sales volume.
 3. Find the average sales per month over the last year.
 4. Calculate the percentage of total sales for each product.
 5. Find the difference in sales between the current month and the previous month.

Super Advanced SQL Problems

Here are five advanced SQL problems designed to challenge and deepen your understanding of SQL:

1. **Product Performance Analysis:** Analyze sales performance over five years and identify the top 10 products with the highest percentage growth in units sold.
 2. **Customer Lifetime Value (CLV) Calculation:** Calculate CLV based on sales data, considering purchase frequency and value.
 3. **Advanced Warranty Claim Trends:** Analyze trends in warranty claims compared to product sales over the past two years.
 4. **Market Basket Analysis:** Perform a market basket analysis to find products frequently purchased together.
 5. **Predictive Sales Modeling:** Predict next month's sales using historical data and a linear regression model.
-